

SKIN CANCER DETECTION

Project Report

Machine Learning & Deep Learning Based Image Classification

HAM10000 Dataset | VGG19 vs. E-VGG19 | SVM | KNN | CNN Architectures

PART I

Project Proposal & Introduction

Members

Shan Ahmed	Maryam Hasnat	Fajr Asim Butt
S2024266154	S2024266194	S2024266039

1. Introduction

1.1 Research Background

Skin cancer is one of the most common forms of cancer worldwide and remains a major public health concern because delayed diagnosis can lead to disease progression, more invasive treatment, and poorer patient outcomes. Melanoma is the most aggressive form, while basal cell carcinoma and squamous cell carcinoma are also highly prevalent and place a substantial burden on healthcare systems. Recent global evidence shows that melanoma incidence remains significant, and ultraviolet radiation continues to be a dominant risk factor in a large share of cases. More broadly, cancer control organizations continue to emphasize that early detection is one of the most effective ways to reduce the burden of cancer through timely diagnosis and treatment.

In dermatology, accurate identification of suspicious skin lesions is challenging because lesion appearance can vary considerably in color, texture, border shape, and size, and benign lesions may visually resemble malignant ones. For this reason, computer-aided diagnosis has gained increasing attention, especially through machine learning and deep learning methods that analyze dermoscopic or clinical images to support classification. Machine Learning techniques have shown promise in analyzing medical imaging data, such as dermoscopic images and histopathological slides, to aid clinicians in making more informed diagnostic decisions. Machine Learning Algorithms can learn complex patterns and relationships from large datasets, potentially enhancing the accuracy and efficiency of skin cancer diagnosis. The aim of this project is to build a robust machine learning model which can analyze images of lesions and detect cancer from images.

Recent studies also show that the field is moving beyond traditional classifiers toward more advanced deep learning frameworks. The 2026 Biomedical Signal Processing and Control paper showed that advanced deep learning models can better capture lesion characteristics and improve sensitivity and specificity in skin cancer analysis. This indicates that skin cancer detection is no longer only a medical imaging problem, but also an important applied machine learning problem where model design, data quality, and clinical relevance must be addressed together.

1.2 Problem Statement

Despite the progress of machine learning and deep learning in medical image analysis, accurate skin cancer detection remains a difficult problem. Misclassification between benign and malignant lesions can have serious clinical consequences, including delayed treatment for dangerous lesions or unnecessary anxiety and intervention for harmless ones. The complexity of lesion appearance, inter-class similarity, image quality variation, and diagnostic subjectivity all contribute to this challenge. Systematic reviews in the literature explicitly note that skin cancer diagnosis requires both early detection and high classification accuracy, yet reliable performance is still difficult to achieve consistently across studies.

Another practical problem is that many published models report strong results in controlled experimental settings but do not always demonstrate robustness for broader real-world use. Performance may drop when models are applied to imbalanced datasets, diverse skin tones, different acquisition settings, or multiclass lesion categories. As a result, there is still a need for systems that are not only accurate in theory but also generalizable, clinically meaningful, and capable of supporting early and dependable decision-making.

1.3 Research Gaps

The first gap: Articles reviewed have focused either on reviewing existing machine learning techniques or on proposing high-performing deep learning architectures, but fewer studies fully address the gap between experimental accuracy and practical applicability. Many models differ in accuracy, precision, and recall, with some metrics high while others remain low. These models are not as beneficial in medical fields where consistent performance across all metrics is crucial. Deep Learning models can achieve better results compared to Machine Learning models but they usually have very complicated architectures.

A second gap is the limited integration of methodological issues that directly affect trustworthy deployment, such as data imbalance, multiclass discrimination, variation in lesion presentation, and reproducibility across datasets. Recent work in 2026 suggests that newer deep learning frameworks can improve predictive performance, but the continued appearance of new architectures itself reflects that existing methods still have unresolved weaknesses.

A third gap is that much of the literature emphasizes model performance metrics without equally addressing how the system can be framed for actual clinical assistance, especially in settings where rapid screening, limited specialist access, or early triage are important. Therefore, this project is justified in aiming to contribute a more reliable skin cancer detection approach that can support early diagnosis and improve the consistency of lesion classification.

1.4 Aim of the Project

The aim of this project is to develop and evaluate an intelligent skin cancer detection framework using machine learning and deep learning-based image classification models for the early identification of malignant skin lesions. The project is primarily focused on supporting early-stage cancer detection, where prompt diagnosis can significantly improve clinical decision-making, treatment planning, and patient outcomes.

This study aims to construct, train, and compare multiple classification models on dermoscopic skin lesion data in order to determine the most effective approach for accurate lesion categorization. Special emphasis will be placed on Convolutional Neural Network (CNN)-based architectures because of their strong capability in automatically extracting hierarchical visual features such as texture, shape, border irregularity, and color variation from medical images.

In addition to standard CNN-based models, the project also aims to build and evaluate an Enhanced VGG19 (E-VGG19) model. The inclusion of this model is motivated by findings from recent research literature, where VGG19-based architectures have shown promising performance in medical image classification tasks. The performance of the E-VGG19 model will be experimentally assessed and compared with other developed models.

Overall, the project aims not only to detect skin cancer at an early stage but also to perform a systematic performance evaluation of different machine learning and CNN-based models, including the proposed E-VGG19 model, in order to identify the most accurate, reliable, and clinically meaningful classification approach for skin lesion analysis.

2. Methodology (Research Design)

This research will adopt an experimental and comparative methodology to develop an automated skin cancer detection system using machine learning and deep learning-based image classification models. The study will be carried out in a sequence of stages, including dataset selection, image preprocessing, model development, training, testing, performance evaluation, and comparative analysis.

2.1 Research Design

The project will use a quantitative, model-development research design. The main objective is to build several classification models for early skin cancer detection and then compare their performance on the same dataset under the same evaluation conditions. The study will focus on image-based classification, where dermoscopic skin lesion images will be used as input and lesion classes will be predicted as output.

2.2 Dataset Selection

The dataset for this study will be selected from publicly available and widely used skin lesion image repositories, primarily ISIC and HAM10000. The ISIC Archive is an established dermatology imaging resource created to improve melanoma early detection, and it provides curated skin lesion images with quality control and expert-supported metadata.

The HAM10000 dataset is also highly suitable for this research because it is a large, multi-source dermoscopic image dataset containing 10,015 images of common pigmented skin lesions and has been extensively used in skin lesion classification studies. Depending on the final scope of the project, the study may use HAM10000 as the primary multiclass dataset, or an ISIC challenge dataset for additional validation or benchmarking.

2.3 Tools and Implementation Environment

The models will be implemented using a standard machine learning and deep learning development environment. The expected tools include:

- Python as the main programming language
- TensorFlow or PyTorch for deep learning model implementation
- TensorFlow.keras.preprocessing.images for Image augmentation of DEEP Learning models
- Scikit-learn for preprocessing support and evaluation utilities
- NumPy, Pandas, and Matplotlib, Seaborn for data handling, analysis, and visualization
- VS Code with WSL Ubuntu for Virtual Environment to use GPU locally

3. Exploratory Data Analysis & Metadata Processing

The HAM10000 dataset ("Human Against Machine with 10,000 Training Images") comprises 10,015 high-resolution dermoscopic images of skin lesions, accompanied by a structured metadata file

(HAM10000_metadata.csv). Prior to model development, a thorough exploratory analysis of both the imagery and the associated metadata was conducted to inform all subsequent design decisions.

3.1 Disease Classes and Binary Target Mapping

The metadata includes a diagnostic label column (dx) covering seven distinct dermatological classes. To frame this as a clinically meaningful binary classification problem, these classes were consolidated into a single binary target variable, where 0 denotes a benign lesion and 1 denotes a malignant or pre-cancerous lesion, as detailed in the table below.

Table 1: Diagnostic Class Mapping to Binary Target Variable

Code	Disease Name	Category	Binary Target
mel	Melanoma	Cancerous	Malignant (1)
bcc	Basal Cell Carcinoma	Cancerous	Malignant (1)
akiec	Actinic Keratoses / Intraepithelial Carcinoma	Pre-cancerous	Malignant (1)
nv	Melanocytic Nevi	Non-cancerous	Benign (0)
bkl	Benign Keratosis-like Lesions	Non-cancerous	Benign (0)
vasc	Vascular Lesions	Non-cancerous	Benign (0)
df	Dermatofibroma	Non-cancerous	Benign (0)

3.2 Resolution of Missing Values

During the statistical profiling of the dataset, 57 missing values (NaNs) were identified in the patient demographic column age. Rather than discarding these records — which would reduce the effective training sample size — the missing values were imputed using the dataset's median age (approximately 39 years). This approach preserves sample diversity while avoiding the introduction of systematic bias that mean imputation could introduce in the presence of outliers.

3.3 Image Path Mapping

The raw image data is distributed across two separate directory partitions: HAM10000_images_part_1 and HAM10000_images_part_2. The preprocessing pipeline recursively parses both directories, matches each file's base name against the image_id column in the metadata file, and appends a direct filesystem path column to the dataframe. Any metadata entries for which no corresponding image file could be located are subsequently removed, ensuring full alignment between metadata records and available imagery.

Dataset Google Drive Link:

https://drive.google.com/drive/folders/1canWKji2hzc0QjNLrnSCWRngyXemgWiQ?usp=drive_link

3.4 Dataset Splitting Strategy

A robust, leakage-free evaluation framework is essential for producing reliable performance estimates. The following strategy was adopted to ensure the integrity of all three data partitions.

- **Splitting Ratio:** The full dataset is partitioned into 70% Training, 15% Validation, and 15% Testing subsets.
- **Stratified Splitting:** Given the pronounced class imbalance in the dataset (approximately 85.7% Benign versus 14.3% Malignant), stratified splitting was applied based on the binary target variable. This guarantees that the class distribution is preserved identically across all three subsets, preventing any partition from being disproportionately dominated by one class.

This approach ensures that the model is evaluated on held-out data whose class proportions accurately reflect those of the full dataset, thereby yielding unbiased estimates of generalization performance.

4. Pipeline for Traditional Machine Learning

Classical machine learning algorithms are not natively equipped to process high-dimensional raw color images. Accordingly, a dedicated preprocessing and feature extraction pipeline was developed to transform each image into a structured numerical representation suitable for these classifiers.

4.1 Step-by-Step Preprocessing Pipeline

Figure 1: Traditional ML preprocessing pipeline



Each stage of the pipeline is described in detail below:

- **Sampling:** A representative subset of 3,000 images is selected from the full dataset to maintain computational feasibility, particularly for resource-intensive classifiers such as the Support Vector Machine.
- **Grayscale Conversion:** Each image is converted to single-channel grayscale (L mode). Since classical classifiers rely primarily on texture and structural features rather than chromatic information, this conversion reduces feature dimensionality by a factor of three with minimal loss of discriminative signal.
- **Downsampling:** Images are resized to 64×64 pixels using Lanczos interpolation, a high-quality anti-aliasing filter that minimizes artefact introduction during downscaling.
- **Flattening:** The 64×64 -pixel grid is unrolled into a one-dimensional feature vector of length 4,096, providing a structured tabular input for the classifiers.
- **Feature Scaling (Z-score Normalization):** The feature vectors are standardized using StandardScaler such that each feature has a mean of zero and a standard deviation of one, following the formula $z = (x - \mu) / \sigma$. This step is critical for distance-based algorithms such as KNN and SVM, which are sensitive to differences in feature scale.

4.2 Model Development

The proposed study will develop and compare multiple classification models for skin cancer detection across two major categories: classical machine learning algorithms and deep learning CNN-based architectures. Each model brings a distinct computational approach, inductive bias, and suitability profile for dermatoscopic image classification. The sections below describe each model in detail.

A. Classical Machine Learning Models

Classical machine learning models require a structured numerical input and cannot directly process raw image pixels. For this project, each image is converted to grayscale, resized to 64×64 pixels, flattened into a 4,096-dimensional vector, and standardized using Z-score normalization before being passed to these classifiers. Each model described below serves as either a baseline or a standalone classifier within the experimental pipeline.

Support Vector Machine — RBF Kernel (SVM-RBF)

The Support Vector Machine with a Radial Basis Function (RBF) kernel is a powerful supervised learning algorithm grounded in statistical learning theory. SVM operates by finding the optimal separating hyperplane — the decision boundary that maximizes the margin between the two classes (Benign and Malignant) in the feature space. The data points that lie closest to this hyperplane are called support vectors, and the entire model is defined by them.

The RBF kernel allows SVM to handle non-linearly separable data by implicitly projecting the input features into a higher-dimensional space where a linear boundary becomes feasible. The kernel function is defined as $K(x, x') = \exp(-\gamma \|x - x'\|^2)$, where γ controls the influence radius of each training point. A small γ means a smooth boundary, while a large γ makes the boundary more complex and data-specific.

In the context of skin cancer detection, the preprocessed 4,096-dimensional greyscale feature vectors are passed to the SVM-RBF classifier. The algorithm searches for the hyperplane that separates benign and malignant feature representations with maximum margin, making it robust to moderate noise and suitable for high-dimensional inputs. SVM-RBF is particularly valued in medical imaging for its strong generalisation on small-to-medium datasets.

Table: SVM-RBF Configuration Summary

Parameter	Value / Description
Kernel	Radial Basis Function (RBF)
Regularization (C)	Tuned via cross-validation (default: 1.0)
Kernel Coefficient (γ)	Scale ($1 / n_features \times \text{variance}$)
Input Features	4,096-dim Z-score normalised grayscale vector
Task	Binary Classification: Benign (0) vs. Malignant (1)
Key Strength	Non-linear boundaries; strong in high-dimensional space

Architecture Diagram — SVM-RBF

Component	Description
Input	4,096-dim Z-score normalized grayscale feature vector
Kernel Mapping	RBF: $K(x, x') = \exp(-\gamma \ x - x'\ ^2) \rightarrow$ implicit high-dimensional feature space
Margin Optimization	Find hyperplane that maximizes margin between Benign and Malignant classes
Support Vectors	Training points closest to decision boundary — define the entire model
Regularization	Parameter C balances margin width vs. misclassification tolerance
Kernel Coefficient	$\gamma = \text{scale} = 1 / (\text{n_features} * \text{variance})$
Decision	$\text{sign}(w^T \phi(x) + b) \rightarrow$ Benign (0) or Malignant (1)

Results SVM-RBF

Metric	Benign	Malignant
Accuracy	72%	72%
Precision	88%	37%
Recall (Sensitivity)	75%	59%
F1-Score	81%	45%

Linear Support Vector Machine (Linear SVM)

The Linear SVM is a variant of the Support Vector Machine that constrains the decision boundary to be strictly linear in the original feature space. Unlike the RBF kernel variant, no implicit mapping into a higher-dimensional space is performed. Instead, the model directly learns a linear hyperplane $w^T x + b = 0$ that maximizes the margin between the two classes.

Linear SVM is computationally more efficient than kernel-based SVMs, particularly for high-dimensional feature spaces where the number of features may exceed the number of samples. In such regimes, the data may already be approximately linearly separable, rendering a complex kernel unnecessary. Linear SVM serves as an important ablation baseline in this study: if it performs comparably to SVM-RBF, it suggests the extracted feature space is largely linearly separable; if it underperforms significantly, it confirms that non-linear boundaries are needed.

For skin lesion classification, Linear SVM provides an interpretable decision boundary, and its training time scales favorably with dataset size, making it a pragmatic choice for evaluating whether complex kernels offer meaningful gains over simpler models.

Table: Linear SVM Configuration Summary

Parameter	Value / Description
Kernel	Linear (no kernel mapping)

Parameter	Value / Description
Regularization (C)	1.0 (default); controls margin width vs. misclassification tolerance
Solver	LibLinear (optimized for large linear problems)
Input Features	4,096-dim Z-score normalized grayscale vector
Role in Study	Ablation baseline to assess linear separability
Key Strength	Fast training; efficient on high-dimensional input

Architecture Diagram — Linear SVM

Component	Description
Input	4,096-dim Z-score normalized grayscale feature vector
Decision Boundary	Linear hyperplane: $w^T x + b = 0$ directly in original feature space (no kernel mapping)
Margin Optimization	Maximize $2/\ w\ $ subject to correct classification of training points
Solver	LibLinear — optimized for large-scale linear problems
Regularization	Parameter C penalises margin violations (slack variables)
Decision	$\text{sign}(w^T x + b) \rightarrow \text{Benign (0) or Malignant (1)}$

Results Linear SVM

Metric	Benign Result	Malignant
Accuracy	75%	75%
Precision	84%	36%
Recall (Sensitivity)	85%	33%
F1-Score	85%	34%

Logistic Regression (L2-Regularised)

Logistic Regression is a classical statistical classification model that estimates the probability that a given input belongs to a particular class. Despite its name, it is a classification algorithm rather than a regression model. It models the conditional probability $P(y=1|x)$ using the logistic (sigmoid) function: $P(y=1|x) = 1 / (1 + \exp(-wTx))$, where w is the weight vector learned during training.

The model is trained by minimizing the binary cross-entropy loss function with L2 regularization (Ridge penalty), which penalizes large weight values and prevents overfitting. The L2 penalty adds a term $\lambda \|w\|^2$ to the loss, encouraging the model to find a solution where feature weights are small and distributed rather than concentrated on a few features.

Logistic Regression is one of the most interpretable classifiers available: each feature's weight directly reflects its contribution to the log-odds of malignancy. In the skin cancer detection pipeline, it serves as a strong probabilistic baseline, providing calibrated probability scores that are clinically meaningful. A predicted probability of 0.85 can be interpreted as the model assigning an 85% likelihood of malignancy, which is directly actionable in a triage context.

Table: Logistic Regression Configuration Summary

Parameter	Value / Description
Regularization Type	L2 (Ridge)
Solver	lbfgs / liblinear
Maximum Iterations	1000
Output	Calibrated probability score $\in [0, 1]$
Input Features	4,096-dim Z-score normalized grayscale vector
Key Strength	Interpretable; calibrated probabilities; fast training

Architecture Diagram — Logistic Regression (L2-Regularised)

Component	Description
Input	4,096-dim Z-score normalized grayscale feature vector
Linear Combination	$z = w^T x + b$ (weighted sum of all input features)
Activation Function	Sigmoid: $P(y=1 x) = 1 / (1 + \exp(-z)) \rightarrow$ probability in $[0, 1]$
Loss Function	Binary Cross-Entropy: $L = -[y * \log(p) + (1-y) * \log(1-p)]$
Regularization	L2 Ridge penalty: $+\lambda w ^2$ added to loss — encourages small, distributed weights
Optimizer	lbfgs / liblinear solver, max 1,000 iterations
Decision	$p \geq 0.5 \rightarrow$ Malignant (1); $p < 0.5 \rightarrow$ Benign (0)

Results Logistic Regression

Metric	Benign Result	Malignant
Accuracy	75%	75%
Precision	84%	35%
Recall (Sensitivity)	85%	34%
F1-Score	84%	34%

K-Nearest Neighbors (KNN, $k = 5$)

K-Nearest Neighbors is a non-parametric, instance-based learning algorithm that makes predictions by examining the k most similar training examples to a query point. During inference, the algorithm computes the Euclidean distance between the test instance and all training samples in the feature space, identifies the k nearest neighbors, and assigns the class label by majority vote among those neighbors.

KNN is termed a 'lazy learner' because it performs no explicit training phase — the entire training dataset is retained and used at inference time. This makes training instantaneous but prediction computationally expensive for large datasets, as distance computations must be performed against the full training set for every query. With $k = 5$, the algorithm uses five nearest neighbors for each classification decision, providing a degree of smoothing that reduces sensitivity to noise compared with $k = 1$.

In the skin lesion classification context, KNN operates on the 4,096-dimensional flattened grayscale feature vectors. Its primary role is to evaluate whether simple proximity-based reasoning in pixel space is sufficient to distinguish benign from malignant lesion representations. KNN is also known to be insensitive to outliers and makes no parametric assumptions about the data distribution, making it a useful non-parametric reference point.

Table: KNN Configuration Summary

Parameter	Value / Description
Number of Neighbors (k)	5
Distance Metric	Euclidean distance
Voting Strategy	Majority vote among k neighbors
Training Paradigm	Lazy learning (no explicit training phase)
Input Features	4,096-dim Z-score normalized grayscale vector
Key Strength	No distribution assumptions; simple and interpretable

Architecture Diagram — K-Nearest Neighbors ($k = 5$)

Component	Description
Input	4,096-dim Z-score normalized grayscale feature vector
Training Phase	Lazy learning — entire training set stored; no explicit model fitting
Distance Metric	Euclidean distance computed to all training samples for each test query
Neighbor Retrieval	For each test sample, find 5 nearest training points by distance
Voting Strategy	Majority vote among 5 neighbors determines predicted class
Decision	Majority class among neighbors → Benign (0) or Malignant (1)

Results KNN (k=5)

Metric	Benign Result	Malignant
Accuracy	81%	81%
Precision	81%	68%
Recall (Sensitivity)	98%	9%
F1-Score	89%	16%

Decision Tree (Maximum Depth = 10)

A Decision Tree is a hierarchical, tree-structured classification model that recursively partitions the input feature space using a series of binary decisions (if-then rules). At each internal node, the algorithm selects the feature and threshold that best separates the data according to a splitting criterion. In this implementation, Gini impurity is used as the criterion, which measures the probability of incorrectly classifying a randomly chosen element if it were classified according to the class distribution at that node.

The Gini impurity for a node is defined as: $Gini = 1 - \sum p_i^2$, where p_i is the proportion of samples belonging to class i at that node. The algorithm greedily selects the split that minimizes the weighted average Gini impurity across the resulting child nodes. This process continues recursively until either the maximum depth is reached, all samples at a node belong to a single class, or no further improvement is possible.

The maximum depth is constrained to 10 in this study, which limits the tree's complexity and reduces the risk of overfitting. Without depth constraints, decision trees tend to grow arbitrarily deep and memorize training data noise. A depth of 10 allows the model to capture meaningful non-linear relationships and interaction effects between features while remaining regularized. Decision trees also offer full model transparency: the learned tree can be visualized and each decision path can be traced, making it uniquely interpretable among the classifiers evaluated in this project.

Table: Decision Tree Configuration Summary

Parameter	Value / Description
Splitting Criterion	Gini Impurity
Maximum Tree Depth	10
Splitting Strategy	Best split (greedy, exhaustive feature search)
Minimum Samples per Leaf	1 (default)
Input Features	4,096-dim Z-score normalized grayscale vector
Key Strength	Fully interpretable; non-linear boundaries; no scaling needed

Architecture Diagram — Decision Tree (Max Depth = 10)

Component	Description
Input	4,096-dim Z-score normalized grayscale feature vector
Splitting Criterion	Gini Impurity: $Gini = 1 - \sum(\pi^2)$ — minimized greedily at each node
Node Structure	Each internal node: feature threshold test → left (True) or right (False) branch
Tree Depth	Maximum depth = 10 — prevents overfitting and memorization of training noise
Leaf Nodes	Terminal nodes assign class by majority label of samples at that node
Visualization	Full decision tree can be plotted — each path is an explicit if-then rule
Decision	Traverse from root to leaf following threshold conditions → Benign (0) or Malignant (1)

Results Decision Tree

Metric	Benign Result	Malignant
Accuracy	68%	68%
Precision	83%	28%
Recall (Sensitivity)	75%	39%
F1-Score	79%	32%

5. Pipeline for Deep Learning via Transfer Learning

Deep Convolutional Neural Networks (CNNs) exploit the full spatial structure of two-dimensional RGB images, rendering the flattening pipeline unnecessary. Transfer learning from models pre-trained on ImageNet is leveraged to obtain rich, semantically meaningful feature representations while drastically reducing the labelled data requirements.

5.1 Real-Time Data Augmentation

To mitigate the combined effects of class imbalance and overfitting onto the pre-trained feature representations, stochastic real-time image augmentation is applied on-the-fly during training using ImageDataGenerator. The following augmentation operations are applied:

- **Rescaling:** Pixel intensity values are normalized from the range $[0, 255]$ to $[0.0, 1.0]$.
- **Rotation:** Random rotations within a range of $\pm 20^\circ$.
- **Translation:** Width and height shifts of up to $\pm 10\%$ of the respective image dimension.
- **Zoom:** Random zooming within a range of $\pm 10\%$.

- **Shear:** Shear transformations within a range of $\pm 5^\circ$.
- **Flipping:** Independent horizontal and vertical flips, both enabled.

5.2 Pre-trained Backbones and Custom Classification Head

Five advanced CNN architectures, each pre-trained on the ImageNet large-scale recognition benchmark, are employed as frozen feature extractors. Their convolutional weights are held fixed while a custom classification head is appended and trained on the HAM10000 skin lesion data. The table below summarizes the key properties of each backbone evaluated.

Table: Pre-trained Backbone Comparison

Architecture	Year	Depth	Parameters	Key Innovation
VGG19	2014	19 layers	~143.67M	Deep uniform 3x3 conv stack; large dense head
ResNet50	2015	50 layers	~25.6M	Residual skip connections ($y = F(x) + x$)
ResNet152V2	2016	152 layers	~60.2M	Pre-activation residual blocks for smooth gradient flow
InceptionV3	2016	~48 layers	~23.9M	Parallel multi-scale Inception modules
DenseNet201	2017	201 layers	~20.2M	Dense connectivity: every layer receives all prior outputs

The following architectures are evaluated:

The unified architecture of the classification head appended to each backbone is as follows:

Figure 2: Custom classification head architecture appended to each pre-trained backbone

Layer / Component	Details
Pre-trained Frozen Backbone	VGG19 / ResNet50 / ResNet152V2 / InceptionV3 / DenseNet201
GlobalAveragePooling2D	Spatial aggregation over feature maps
Dense (256, ReLU)	Fully-connected feature transformation
Batch Normalization	Stabilizes and regularizes activations
Dropout (40%)	Stochastic regularization to reduce overfitting
Dense (1, Sigmoid)	Binary output — probability of malignancy

Training is governed by the following configuration:

- Optimizer: Adam with a learning rate of 1×10^{-4}
- Loss Function: Binary Cross-Entropy

- **Callbacks — ReduceLROnPlateau:** Halves the learning rate if validation loss fails to improve for three consecutive epochs.
- **Callbacks — EarlyStopping:** Terminates training if validation loss does not improve over seven consecutive epochs, restoring the weights from the epoch that achieved the lowest validation loss.

VGG19

VGG19 is a deep convolutional neural network developed by the Visual Geometry Group at the University of Oxford and introduced at the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014. The architecture is characterized by its extreme depth and deliberate simplicity: it uses only 3×3 convolutional filters with a stride of 1 and 2×2 max-pooling layers with a stride of 2, applied consistently throughout the entire network.

The architecture is named for its 19 weight layers — 16 convolutional layers and 3 fully connected layers — organized into five convolutional blocks. Each block groups consecutive convolutional layers before a max-pooling down sampling step. The number of filters doubles with each block: Block 1 uses 64 filters, Block 2 uses 128, and Blocks 3, 4, and 5 each use 256 or 512. This progressive widening of filter banks allows the network to learn increasingly complex and abstract features at deeper layers.

The original VGG19 classification head consists of two fully connected layers of 4,096 units each, followed by a 1,000-unit SoftMax output for ImageNet classification. This head contributes over 123 million parameters — the dominant source of the model's total 143.67 million parameters. Despite its high parameter count, VGG19 remains widely used as a feature extractor in transfer learning for medical imaging tasks due to its rich, semantically meaningful representations and the regularity of its architecture.

Table: VGG19 Architecture Summary

Property	Details
Introduced	Simonyan & Zisserman, 2014 (ILSVRC)
Total Layers	19 weight layers (16 Conv + 3 Dense)
Convolutional Blocks	5 blocks with 3×3 filters
Total Parameters	~143.67 million
Input Size	224 × 224 × 3
Pre-training Dataset	ImageNet (1.28M images, 1,000 classes)
Key Characteristic	Deep, uniform architecture; large classification head
Role in Project	Baseline deep learning model; also, foundation for E-VGG19

Architecture Diagram — VGG19 (Transfer Learning, Frozen Backbone)

Layer / Block	Details
Input	224 x 224 x 3 RGB image, pixel values normalized to [0, 1]
Block 1	Conv2D(64) x 2 + MaxPool2D — Output: 112 x 112 x 64
Block 2	Conv2D(128) x 2 + MaxPool2D — Output: 56 x 56 x 128

Block 3	Conv2D(256) x 4 + MaxPool2D — Output: 28 x 28 x 256
Block 4	Conv2D(512) x 4 + MaxPool2D — Output: 14 x 14 x 512
Block 5	Conv2D(512) x 4 + MaxPool2D — Output: 7 x 7 x 512 [Frozen Backbone End]
All Conv Filters	3x3 filters, stride 1, ReLU activation throughout
Custom Head	GlobalAveragePooling2D → Dense(256, ReLU) → BatchNorm → Dropout(40%) → Dense(1, Sigmoid)
Total Parameters	~143.67M (backbone frozen; custom head trained on HAM10000)
Pre-training	ImageNet (1.28M images, 1,000 classes)

Results VGG19

Metric	Benign Result	Malignant
Accuracy	74%	74%
Precision	94%	41%
Recall (Sensitivity)	73%	79%
F1-Score	82%	54%

ResNet50

ResNet50 (Residual Network with 50 layers) was introduced by He et al. at Microsoft Research in 2015 and won the ILSVRC 2015 competition. The defining innovation of ResNet is the residual connection (also called a skip connection), which creates a shortcut path that adds the input of a block directly to its output: $y = F(x) + x$, where $F(x)$ represents the learned residual mapping. This additive identity shortcut allows gradients to flow directly through the network without passing through non-linear transformations, effectively solving the vanishing gradient problem that had prevented training of very deep networks.

ResNet50 uses a bottleneck block design to manage computational cost at 50 layers. Each bottleneck block contains three convolutional layers: a 1×1 convolution that reduces channel dimensionality, a 3×3 convolution that performs the main spatial feature extraction, and a second 1×1 convolution that restores the original channel depth. This bottleneck structure enables depth without a proportional increase in parameters or FLOPs.

The architecture begins with a 7×7 convolutional stem followed by max-pooling, then four residual stages containing progressively wider feature maps (64, 128, 256, 512 channels). Global Average Pooling is applied before the fully connected output layer, eliminating the large dense flattening layers that dominate VGG19's parameter count. ResNet50's combination of depth, residual connections, and efficient bottleneck blocks makes it one of the most versatile and widely deployed architectures in transfer learning for medical image analysis.

Table: ResNet50 Architecture Summary

Property	Details
Introduced	He et al., Microsoft Research, 2015 (ILSVRC Winner)
Total Layers	50 (using bottleneck blocks)
Key Innovation	Residual (skip) connections: $y = F(x) + x$
Block Type	Bottleneck: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ Conv
Total Parameters	~25.6 million
Input Size	$224 \times 224 \times 3$
Head Design	Global Average Pooling (no large dense layers)
Key Strength	Solves vanishing gradient; deep but efficient

Architecture Diagram — ResNet50 (Transfer Learning, Frozen Backbone)

Layer / Block	Details
Input	$224 \times 224 \times 3$ RGB image, pixel values normalized to [0, 1]
Stem	Conv2D(64, 7x7, stride 2) + BatchNorm + ReLU + MaxPool — Output: $56 \times 56 \times 64$
Stage 1	3 Bottleneck blocks ($1 \times 1 \rightarrow 64$, $3 \times 3 \rightarrow 64$, $1 \times 1 \rightarrow 256$) + residual skip — Output: $56 \times 56 \times 256$
Stage 2	4 Bottleneck blocks (128 channels) + residual skip — Output: $28 \times 28 \times 512$
Stage 3	6 Bottleneck blocks (256 channels) + residual skip — Output: $14 \times 14 \times 1024$
Stage 4	3 Bottleneck blocks (512 channels) + residual skip — Output: $7 \times 7 \times 2048$ [Frozen]
Skip Connection	$y = F(x) + x$ — identity shortcut added to each bottleneck block output
Bottleneck Design	1×1 Conv (reduce) $\rightarrow 3 \times 3$ Conv (extract) $\rightarrow 1 \times 1$ Conv (restore) per block
Custom Head	GlobalAveragePooling2D $\rightarrow$ Dense(256, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(40%) $\rightarrow$ Dense(1, Sigmoid)
Total Parameters	~25.6M total; ~23.5M frozen backbone

Results ResNet50

Metric	Benign Result	Malignant
Accuracy	81%	82%
Precision	85%	55%
Recall (Sensitivity)	93%	33%

F1-Score	89%	41%
----------	-----	-----

ResNet152V2

ResNet152V2 is a significantly deeper variant of the residual network family, containing 152 weight layers, and represents the V2 revision of the original ResNet design by He et al. (2016). The V2 improvement introduces pre-activation residual blocks, where Batch Normalization and ReLU activation are applied before the convolutional operations rather than after them. This ordering change — often referred to as the 'full pre-activation' design — produces smoother gradient flow during backpropagation and empirically improves generalization performance compared with the original post-activation design.

The pre-activation design means that in the identity shortcut path, the input passes directly without any activation applied, making it a true identity mapping. This property was shown by He et al. to be important for enabling very deep networks to converge reliably, as the residual functions can be learned relative to a clean, unmodified identity baseline.

With 152 layers, ResNet152V2 has substantially greater representational capacity than ResNet50, enabling it to learn more abstract and fine-grained feature hierarchies. This additional depth is particularly beneficial for complex visual discrimination tasks such as distinguishing subtle textural and morphological differences between benign and malignant skin lesions. However, the increased depth also raises training time and memory requirements, and the risk of overfitting is higher on smaller datasets, making regularization strategies such as Dropout and Early Stopping especially important when fine-tuning this architecture.

Table: ResNet152V2 Architecture Summary

Property	Details
Introduced	He et al., 2016 (V2 pre-activation variant)
Total Layers	152 weight layers
Key Innovation	Pre-activation blocks (BN → ReLU → Conv)
Block Type	Pre-activation bottleneck residual block
Total Parameters	~60.2 million
Input Size	$224 \times 224 \times 3$
Advantage over ResNet50	Greater depth; better gradient flow via pre-activation
Key Strength	High capacity for complex feature hierarchies

Architecture Diagram — ResNet152V2 (Transfer Learning, Frozen Backbone)

Layer / Block	Details
Input	224 x 224 x 3 RGB image, pixel values normalized to [0, 1]
Stem	Conv2D(64, 7x7, stride 2) + BatchNorm + ReLU + MaxPool — Output: 56 x 56 x 64

Pre-activation Block	BatchNorm → ReLU → Conv (V2 key change: activations before weights)
Stage 1	3 Pre-activation bottleneck blocks — Output: 56 x 56 x 256
Stage 2	8 Pre-activation bottleneck blocks — Output: 28 x 28 x 512
Stage 3	36 Pre-activation bottleneck blocks — Output: 14 x 14 x 1024
Stage 4	3 Pre-activation bottleneck blocks — Output: 7 x 7 x 2048 [Frozen]
Skip Connection	True identity shortcut — input passes directly without any activation applied
Custom Head	GlobalAveragePooling2D → Dense(256, ReLU) → BatchNorm → Dropout(40%) → Dense(1, Sigmoid)
Total Parameters	~60.2M total; deeper representational capacity vs. ResNet50

Results ResNet152V2

Metric	Benign Result	Malignant
Accuracy	81%	82%
Precision	85%	41%
Recall (Sensitivity)	93%	83%
F1-Score	89%	55%

InceptionV3

InceptionV3 is a deep CNN architecture developed by Szegedy et al. at Google and released in 2015–2016 as the third iteration of the Inception family. Its central design philosophy is factorization and multi-scale feature extraction: rather than using a single large convolutional filter, InceptionV3 applies multiple filter sizes in parallel within the same layer and concatenates their outputs, allowing the network to simultaneously capture both fine-grained local textures and broader spatial patterns.

The fundamental computational unit of InceptionV3 is the Inception module. Each module contains parallel branches with different filter configurations — for example, 1×1 convolutions (for dimensionality reduction and cross-channel feature mixing), 3×3 convolutions (for medium-scale features), 5×5 convolutions factorized into two 3×3 operations (for efficiency), and max-pooling branches. The outputs of all branches are concatenated depth-wise into a single feature map tensor.

A key engineering contribution of InceptionV3 is the factorization of large convolutional filters: a 7×7 convolution is replaced with a series of 1×7 and 7×1 asymmetric convolutions, and 5×5 filters are replaced with two stacked 3×3 filters. This reduces computational cost while maintaining the same effective receptive field. InceptionV3 also incorporates auxiliary classifiers during training to improve gradient flow to earlier layers. For skin lesion classification, InceptionV3's multi-scale feature capture is well-suited to dermoscopic images, where diagnostic features may appear at different spatial scales simultaneously.

Table: InceptionV3 Architecture Summary

Property	Details
Introduced	Szegedy et al., Google, 2015–2016
Total Layers	~48 weight layers (deep Inception module stack)
Key Innovation	Parallel multi-scale Inception modules
Filter Factorization	$5 \times 5 \rightarrow$ two 3×3 ; $7 \times 7 \rightarrow 1 \times 7 + 7 \times 1$ (asymmetric)
Total Parameters	~23.9 million
Input Size	$299 \times 299 \times 3$ (resized to 224×224 in this project)
Key Strength	Multi-scale feature capture; computationally efficient
Medical Imaging Benefit	Captures fine texture and broad lesion shape simultaneously

Architecture Diagram — InceptionV3 (Transfer Learning, Frozen Backbone)

Layer / Block	Details
Input	$224 \times 224 \times 3$ RGB image (native input 299×299 , resized in this project)
Stem	3×3 Conv $\times 3$ + Maxol $\rightarrow 3 \times 3$ Conv $\times 2$ + MaxPool — initial feature extraction
Inception Module A	Parallel branches: 1×1 Conv $1 \times 1 + 3 \times 3$ Conv $1 \times 1 + 3 \times 3 + 3 \times 3$ Conv AvgPool + 1×1 — concatenated
Inception Module B	Factorized large filters: 5×5 replaced by $1 \times 7 + 7 \times 1$ asymmetric convolutions
Inception Module C	Expanded: 3×3 branches split into parallel 1×3 and 3×1 for wider representations
Auxiliary Classifiers	Two auxiliary output heads during training to improve gradient flow to early layers
Custom Head (this project)	GlobalAveragePooling2D $\rightarrow$ Dense(256, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(40%) $\rightarrow$ Dense(1, Sigmoid)
Total Parameters	~23.9M total; computationally efficient for its depth

Results InceptionV3

Metric	Benign Result	Malignant
Accuracy	74%	74%
Precision	92%	42%
Recall (Sensitivity)	75%	74%
F1-Score	83%	53%

DenseNet201

DenseNet201 (Densely Connected Convolutional Network with 201 layers) was introduced by Huang et al. at Cornell University in 2017 and represents a fundamental rethinking of inter-layer connectivity in deep networks. In a standard CNN, each layer receives input only from the immediately preceding layer. In ResNet, each layer additionally receives a shortcut from one or two layers back. DenseNet takes this principle to its logical extreme: every layer receives the concatenated feature maps from all preceding layers within its dense block as input, and passes its own output to all subsequent layers.

Formally, in a dense block with L layers, the total number of direct connections is $L(L+1)/2$ rather than L as in a standard network. The output of layer l is: $x_l = Hl([x_0, x_1, \dots, x_{l-1}])$, where $[\cdot]$ denotes concatenation and Hl is a composite function of Batch Normalization, ReLU, and 3×3 convolution. This dense connectivity ensures maximum feature reuse across all layers, meaning early low-level features (edges, textures) remain accessible to later, more abstract layers.

DenseNet201 is organized into four dense blocks separated by transition layers, which apply 1×1 convolution and average pooling to reduce spatial resolution and control feature map growth between blocks. The growth rate hyperparameter k (typically 32) controls how many new feature maps each layer contributes to the concatenated input of subsequent layers. DenseNet's architecture provides strong regularization through feature reuse, as redundant feature learning is suppressed — any feature that is useful can simply be passed forward without being re-learned. This property is especially advantageous for skin cancer classification on HAM10000, where the dataset is moderately sized and overfitting is a genuine concern.

Table: DenseNet201 Architecture Summary

Property	Details
Introduced	Huang et al., Cornell University, 2017
Total Layers	201 weight layers
Key Innovation	Dense connectivity: every layer connects to all subsequent layers
Connectivity	$x_l = Hl([x_0, x_1, \dots, x_{l-1}])$ — concatenation of all prior outputs
Total Parameters	~20.2 million
Input Size	$224 \times 224 \times 3$
Growth Rate (k)	32 new feature maps per layer
Key Strength	Maximum feature reuse; strong implicit regularization; efficient
Medical Imaging Benefit	Retains fine-grained early features throughout the network depth

Architecture Diagram — DenseNet201 (Transfer Learning, Frozen Backbone)

Layer / Block	Details
Input	$224 \times 224 \times 3$ RGB image, pixel values normalized to $[0, 1]$
Stem	Conv2D(64, 7×7 , stride 2) + BatchNorm + ReLU + MaxPool — Output: $56 \times 56 \times 64$

Dense Block 1	6 layers; each layer receives all previous feature maps concatenated as input
Transition Layer 1	1x1 Conv (compression) + Average Pooling — halves spatial dimensions
Dense Block 2	12 layers; $x_l = Hl([x_0, x_1, \dots, x_{l-1}])$ — growth rate $k = 32$ new maps per layer
Transition Layer 2	1x1 Conv + Average Pooling
Dense Block 3	48 layers (deepest block; richest hierarchical feature representations)
Transition Layer 3	1x1 Conv + Average Pooling
Dense Block 4	32 layers — Output: 7 x 7 x 1920 [Frozen Backbone End]
Custom Head	GlobalAveragePooling2D → Dense(256, ReLU) → BatchNorm → Dropout(40%) → Dense(1, Sigmoid)
Total Parameters	~20.2M total; highly parameter-efficient via feature reuse

Results DenseNet201

Metric	Benign Result	Malignant
Accuracy	76%	76%
Precision	91%	43%
Recall (Sensitivity)	78%	69%
F1-Score	84%	53%

6. Proposed Enhanced VGG19 (E-VGG19) Architecture

The central contribution of this project is the implementation and evaluation of an Enhanced VGG19 (E-VGG19) architecture, as proposed by [Irfan Ali Kandhro et al. \(Heliyon, 2024\)](#). This architecture introduces targeted structural modifications to the standard VGG19 backbone in order to reduce parameter redundancy, mitigate the risk of vanishing gradients, and improve convergence behavior on dermoscopic imagery.

6.1 Structural Modifications to VGG19

Three deliberate architectural changes are introduced relative to the standard VGG19 topology:

- **Reduced Convolutional Depth in Block 3:** The third convolutional block is reduced from four to three convolutional layers. This decreases parameter count at the middle stage of the network,

where feature maps are still spatially large, thereby lowering the risk of overfitting and reducing computational cost.

- **Batch Normalization After Pooling:** A Batch Normalization layer is inserted immediately after each MaxPooling2D operation. This regularizes the activation distributions between blocks, accelerates convergence, and reduces the model's sensitivity to the choice of weight initialization.
- **Global Average Pooling Head:** The standard VGG19 densely-connected flattening head — which introduces tens of millions of parameters — is replaced with a single GlobalAveragePooling2D layer. This dramatically reduces the parameter footprint of the network's transition stage while retaining spatial summary statistics.

6.2 Hybrid Classifier Integration

In the first phase of the E-VGG19 evaluation, the network is employed as a static feature extractor within a hybrid modelling framework:

- The E-VGG19 model is initialized by transferring weights from the standard VGG19 pre-trained on ImageNet, applied layer-by-layer wherever layer names match.
- Full images are passed through the network in a single forward pass, and the output of the GlobalAveragePooling2D layer is extracted as a compact 512-dimensional feature vector per image.
- These feature vectors are standardized using StandardScaler and subsequently used as input to three hybrid classifiers: SVM with RBF kernel, Logistic Regression, and Random Forest.

This hybrid approach enables a principled investigation of whether deep features extracted by a purpose-modified backbone offer advantages over the raw pixel features used in the classical pipeline.

6.3 End-to-End Fine-Tuning

In the final and most comprehensive phase of the experimental design, the E-VGG19 backbone is fine-tuned end-to-end. A multi-layer classification head is appended on top of the backbone and all weights — both the backbone and the classification head — are jointly optimized:

Figure 3: E-VGG19 fine-tuning classification head

Layer	Description
Dense (512, ReLU)	High-capacity feature transformation
Batch Normalisation	Activation regularization
Dropout (40%)	Primary stochastic regularization
Dense (256, ReLU)	Secondary feature compression
Batch Normalization	Second-stage regularization
Dropout (30%)	Secondary stochastic regularization
Dense (1, Sigmoid)	Binary output — probability of malignancy

The backbone weights are unfrozen and the entire network is trained end-to-end using a deliberately low learning rate. This allows the convolutional filters to be progressively adapted from their ImageNet-optimized representations towards the specific visual characteristics of dermatoscopic skin lesion imagery, without catastrophically overwriting the pre-trained feature hierarchy.

Architecture Diagram — Enhanced VGG19 (E-VGG19)

Layer / Block	Details
Input	224 x 224 x 3 RGB image, pixel values normalized to [0, 1]
Block 1	Conv2D(64) x 2 + MaxPool2D + BatchNorm — Output: 112 x 112 x 64
Block 2	Conv2D(128) x 2 + MaxPool2D + BatchNorm — Output: 56 x 56 x 128
Block 3 [PRUNED]	Conv2D(256) x 3 + MaxPool2D + BatchNorm — Output: 28 x 28 x 256 (reduced from 4 layers)
Block 4	Conv2D(512) x 4 + MaxPool2D + BatchNorm — Output: 14 x 14 x 512
Block 5	Conv2D(512) x 4 + MaxPool2D + BatchNorm — Output: 7 x 7 x 512 [Frozen Backbone End]
BatchNorm Placement	Inserted after each MaxPool2D — key modification vs. standard VGG19
Feature Vector	GlobalAveragePooling2D → 512-dim compact vector (0 parameters; replaces Flatten+Dense)
Dense Head 1	Dense(512, ReLU) → BatchNorm → Dropout(40%)
Dense Head 2	Dense(256, ReLU) → BatchNorm → Dropout(30%)
Output	Dense(1, Sigmoid) → probability of malignancy in [0, 1]
Total Parameters	~19.83M (vs. 143.67M standard VGG19 — 86.19% reduction, 7.24x compression)
Weight Initialization	Transferred from pretrained VGG19 (ImageNet), layer-by-layer where names match

Results E-VGG19 (End-to-End Fine-Tuned)

Metric	Benign Result	Malignant
Accuracy	80%	81%
Precision	96%	50%
Recall (Sensitivity)	79%	86%
F1-Score	87%	63%

PART II

VGG19 vs. E-VGG19: Architecture & Model Compression Report

7. Executive Summary

This part presents a rigorous comparative analysis between the standard VGG19 convolutional neural network and the proposed Enhanced VGG19 (E-VGG19) architecture, developed in the context of skin lesion classification on the HAM10000 dataset.

Standard VGG19, while highly capable on large-scale recognition benchmarks, carries a substantial computational burden — 143.67 million parameters in total, with the overwhelming majority concentrated in its densely connected classification head. This renders the architecture prone to overfitting on smaller medical imaging datasets and impractical for deployment on resource-constrained hardware.

The E-VGG19 architecture addresses these limitations through three targeted structural interventions: selective pruning of redundant convolutional layers, the systematic integration of Batch Normalization, and the replacement of the dense classification head with a Global Average Pooling layer followed by a compact custom classifier.

Figure 4: Key compression and efficiency metrics — E-VGG19 vs. Standard VGG19

7.24× Overall Compression Ratio	86.19% Total Parameter Reduction	99.68% Classification Head Reduction	~495 MB Disk Space Saved
---	--	--	------------------------------------

These results confirm that E-VGG19 achieves a 7.24× overall compression ratio, reducing the total parameter count from 143.67 million to 19.83 million — an 86.19% reduction — while retaining the representational power of the convolutional backbone. This makes E-VGG19 highly suitable for edge deployment and training on consumer-grade GPUs.

8. Architectural Comparison

The VGG19 backbone is composed of five convolutional blocks, each terminating in a max-pooling operation. E-VGG19 preserves this foundational structure while introducing block-level modifications that reduce depth, enforce activation normalization between blocks, and streamline the output representation.

8.1 Structural Comparison

Table 2 below provides a layer-by-layer comparison of the two architectures, highlighting components of E-VGG19 that differ from their standard VGG19 counterparts.

Table 2: Layer-by-layer structural comparison — Standard VGG19 vs. Proposed E-VGG19

Feature / Layer	Standard VGG19	Proposed E-VGG19
Input Shape	$224 \times 224 \times 3$	$224 \times 224 \times 3$
Block 1	2 Conv (64 filters) + MaxPool	2 Conv (64 filters) + MaxPool + BatchNorm
Block 2	2 Conv (128 filters) + MaxPool	2 Conv (128 filters) + MaxPool + BatchNorm
Block 3	4 Conv (256 filters) + MaxPool	3 Conv (256 filters) + MaxPool + BatchNorm ← Pruned
Block 4	4 Conv (512 filters) + MaxPool	4 Conv (512 filters) + MaxPool + BatchNorm
Block 5	4 Conv (512 filters) + MaxPool	4 Conv (512 filters) + MaxPool + BatchNorm
Feature Extraction	Flatten (25,088-dim)	GlobalAveragePooling2D (512-dim, 0 params)
Classification Head	Dense(4096) → Dense(4096) → Softmax	Dense(512) → Dense(256) → Sigmoid

9. Detailed Parameter and Compression Calculations

To precisely quantify the compression achieved, the parameter counts for both architectures are derived analytically, separated into the convolutional feature extraction backbone and the downstream classification head.

9.1 Feature Extractor Backbone

The convolutional backbone is responsible for learning hierarchical spatial representations of the input image. The standard VGG19 backbone comprises 16 Conv2D layers, with a total frozen parameter count of 20,024,384. E-VGG19 modifies this backbone in two ways:

- **Removal of One Convolutional Layer (Block 3 Pruning):** One Conv2D layer is eliminated from Block 3. This layer operates with 256 input channels, 256 output channels, and a 3×3 kernel. Parameter count removed: $(3 \times 3 \times 256 + 1) \times 256 = 590,080$ parameters removed.
- **Addition of Batch Normalization Layers:** A Batch Normalization layer is inserted after each of the five MaxPooling2D stages. Each such layer learns four parameters per channel (γ , β , moving mean, moving variance).

Table 3: Batch Normalization parameter contributions per block

BN Layer	Channels	Parameters (4 per channel)	Formula
BN 1	64	256	64×4
BN 2	128	512	128×4

BN Layer	Channels	Parameters (4 per channel)	Formula
BN 3	256	1,024	256×4
BN 4	512	2,048	512×4
BN 5	512	2,048	512×4
Total	—	5,888	—

E-VGG19 Backbone Parameter Count: $20,024,384 - 590,080 + 5,888 = 19,440,192$ parameters (-2.92% vs. standard VGG19 backbone). Although the backbone reduction is modest in absolute percentage terms, the addition of Batch Normalization provides substantial training benefits: it regularizes inter-block activation distributions, accelerates convergence, and reduces sensitivity to weight initialization choices.

9.2 Classification Head — Primary Source of Compression

The classification head is the dominant source of parameter overhead in the standard VGG19 architecture, and consequently the primary target of compression in E-VGG19.

Standard VGG19 Classification Head

The standard VGG19 head begins by flattening the $7 \times 7 \times 512$ final feature maps into a 25,088-dimensional vector, followed by three large fully-connected layers:

- Dense Layer 1 — 4,096 units: $(25,088 + 1) \times 4,096 = 102,768,640$ parameters
- Dense Layer 2 — 4,096 units: $(4,096 + 1) \times 4,096 = 16,781,312$ parameters
- Dense Layer 3 — 1,000 units (ImageNet): $(4,096 + 1) \times 1,000 = 4,097,000$ parameters
- Standard VGG19 Head Total: $102,768,640 + 16,781,312 + 4,097,000 = 123,646,952$ parameters

E-VGG19 Binary Classification Head

E-VGG19 replaces the flattening operation with Global Average Pooling (GAP), which computes the spatial mean of each 7×7 feature map, yielding a compact 512-dimensional vector with zero learnable parameters. This is followed by a three-layer compact classification head:

- Dense Layer 1 — 512 units: $(512 + 1) \times 512 = 262,656$ parameters
- Dense Layer 2 — 256 units: $(512 + 1) \times 256 = 131,328$ parameters
- Dense Layer 3 — 1 unit (Binary): $(256 + 1) \times 1 = 257$ parameters
- E-VGG19 Head Total: $262,656 + 131,328 + 257 = 394,241$ parameters (99.68% reduction from standard VGG19 head)

This single architectural substitution — replacing a three-layer dense flattening head with GAP and a compact classifier — eliminates over 123.25 million parameters, representing a 99.68% reduction in classification head size.

10. Total Model Parameter Summary

Table 4: Complete model parameter and size comparison

Model Component	Standard VGG19	Proposed E-VGG19	Parameter Savings
Backbone (Convolutional)	20,024,384	19,440,192	584,192
Classification Head	123,646,952	394,241	123,252,711
Total Parameters	143,671,336	19,834,433	123,836,903
Model Disk Size	~574 MB	~79 MB	~495 MB saved

Overall Compression Summary: Compression Ratio = $143,671,336 / 19,834,433 = 7.24\times$. Total Reduction = $(1 - 19,834,433 / 143,671,336) \times 100 = 86.19\%$ total parameter reduction.

11. Advantages of the E-VGG19 Architecture

The substantial compression achieved by E-VGG19 translates into a set of concrete practical and statistical benefits, each of which is critical in the context of medical imaging on limited hardware.

- **Reduced Overfitting Risk (Structural Regularization):** Standard VGG19's 123-million-parameter classification head presents an extreme overfitting risk when applied to medical imaging datasets, which are typically small relative to natural image corpora. E-VGG19's Global Average Pooling layer and compact classification head act as strong implicit regularizers, drastically curtailing the model's capacity to memorize training samples.
- **GPU Memory Efficiency:** The reduction in total parameter count directly decreases the GPU memory footprint during both forward and backward passes. This eliminates the risk of Out-of-Memory (OOM) errors and enables the full model to be trained on consumer-grade graphics cards — such as the NVIDIA GTX 1660 SUPER — without the need for gradient checkpointing or mixed-precision workarounds.
- **Training Speed and Inference Latency:** With 86% fewer parameters, the number of floating-point operations (FLOPs) required per forward and backward pass is substantially reduced. This results in significantly faster training iterations and lower inference latency, making E-VGG19 viable for real-time diagnostic support applications where response time is a clinical consideration.

In aggregate, these advantages make E-VGG19 a compelling architecture for dermatoscopic skin lesion classification: it retains the proven representational depth of VGG19's convolutional backbone while eliminating the architectural bottlenecks that limit its practical deployment on domain-specific, resource-constrained tasks.

PART III

Dataset Skewness & Sensitivity Optimization

12. Dataset Skewness & Class Imbalance

In clinical diagnostics like skin cancer screening, Sensitivity (Recall) is the most crucial metric. Missing a malignant lesion (a False Negative) has life-threatening consequences, whereas a False Positive only requires a harmless second-level checkup. This section summarizes the dataset skewness, the problems it caused in traditional machine learning and deep learning models, the mathematical solutions implemented, and the final results.

12.1 Dataset Overview and Distribution

The project is trained on the HAM10000 dataset, which exhibits a severe class imbalance across its 10,015 total samples:

- Benign (Class 0): 8,061 samples (80.5%)
- Malignant (Class 1): 1,954 samples (19.5%)

This skewness is visible in the dataset statistics output below, showing distribution by diagnostic code (dx) and data types. The severe imbalance — roughly 4:1 benign to malignant — presents a fundamental challenge for standard classifiers, which tend to optimize accuracy by biasing predictions toward the majority class.

13. Traditional ML Classifiers: Baseline vs. Optimized

13.1 The Baseline Problem

Because of the 80.5% majority class, standard models optimize for overall accuracy by predicting Benign for all cases. This strategy achieves high nominal accuracy while catastrophically failing at malignant detection. The confusion matrices below illustrate this collapse across all five traditional classifiers:

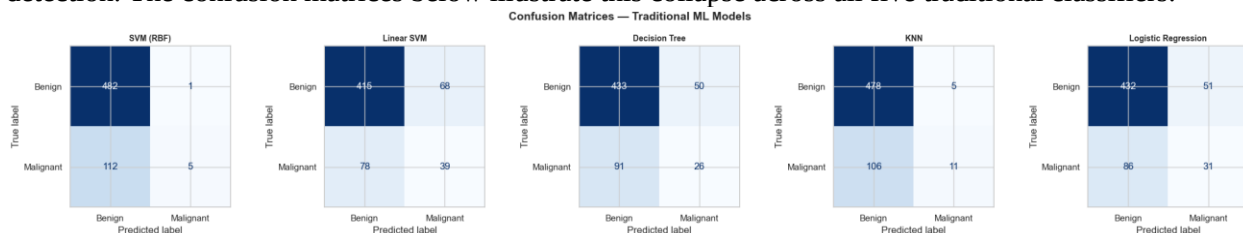
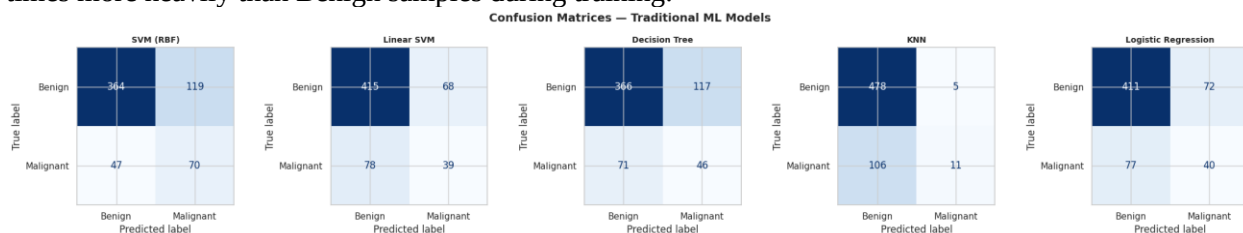


Figure 13.1: Traditional ML Baseline Confusion Matrices (SVM-RBF, Linear SVM, Decision Tree, KNN, Logistic Regression)

- **SVM (RBF) Baseline:** 112 out of 117 malignant cases were missed — only 4.27% Recall.
- **Logistic Regression Baseline:** Malignant Recall was 26.50%.

13.2 The Solution: Class Weight Balancing

The models were updated to use cost-sensitive training via `class_weight='balanced'`. This calculates per-class penalty weights based on inverse frequency, so misclassifying Malignant samples is penalized 4.12 times more heavily than Benign samples during training.



- **SVM (RBF) Balanced:** Malignant Recall rose to 59.83% — a 14x improvement over baseline.
- **Logistic Regression Balanced:** Malignant Recall rose to 33.33% (and 41.88% with a 0.35 decision threshold).

14. Deep Learning Classifiers: Baseline vs. Optimization

14.1 The Baseline Problem

The deep learning models suffered from the same class imbalance problem. VGG19 and ResNet architectures missed large proportions of malignant lesions, while ResNet50 collapsed completely — predicting zero malignant cases across the entire test set:

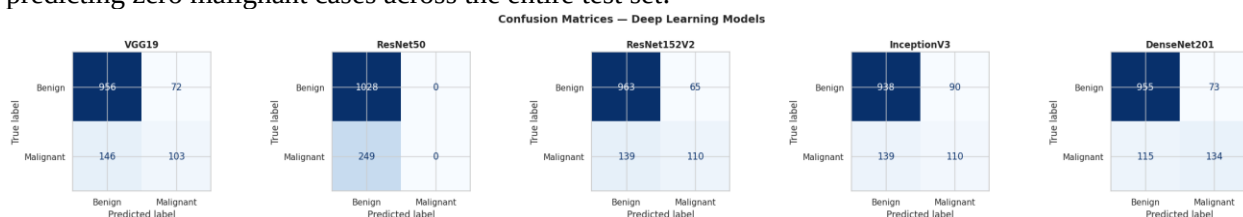
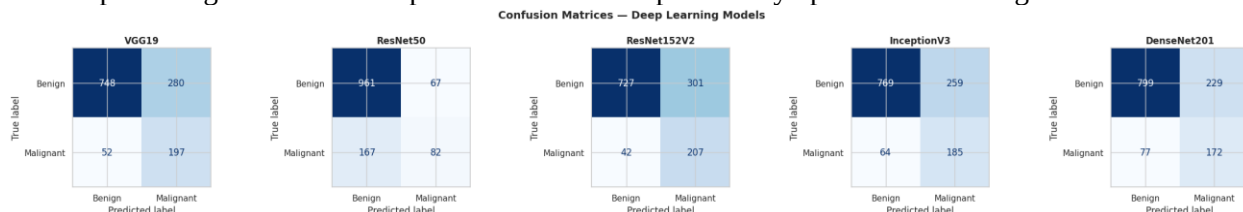


Figure 14.1: Deep Learning Baseline Confusion Matrices (VGG19, ResNet50, ResNet152V2, InceptionV3, DenseNet201)

14.2 The Solution: Cost-Sensitive Loss and Threshold Moving

The deep learning notebook was updated with two complementary optimization strategies:



- **Cost-Sensitive Loss:** Integrated `class_weight` calculation in TensorFlow. Each batch loss is scaled by class weights to prevent models from collapsing to the majority class during training.
- **Threshold Moving (0.35):** Adjusted the classification decision boundary from 0.50 to 0.35, dramatically boosting malignant detection sensitivity by accepting more false positives in exchange for fewer missed cancers.

15. Proposed E-VGG19 Model: Sensitivity Optimization

15.1 Threshold 0.35 Performance

Moving the E-VGG19 decision threshold to 0.35 yields a substantial sensitivity boost. The confusion matrix below demonstrates the model performance at this clinically-motivated threshold:

E-VGG19 End-to-End — Confusion Matrix (Threshold: 0.35)

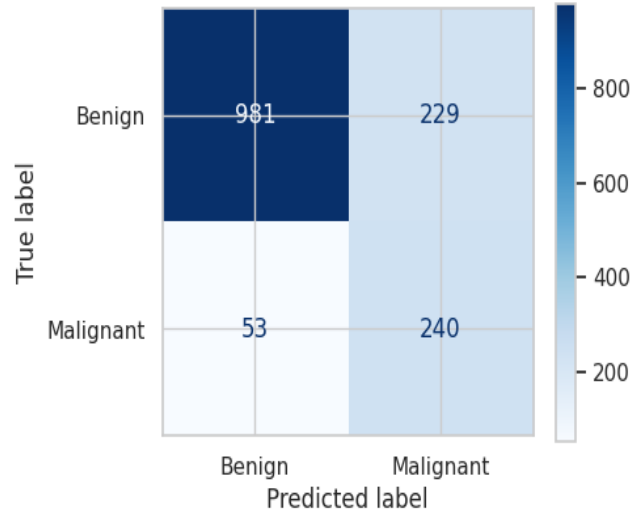


Figure 15.1: E-VGG19 End-to-End Confusion Matrix at Threshold 0.35

- True Benign: 958 correctly classified as Benign, 252 incorrectly flagged as Malignant (False Positives)
- True Malignant: 40 missed as Benign (False Negatives), 253 correctly detected as Malignant (True Positives)
- **Malignant Recall (Sensitivity):** $253 / (253 + 53) = 86.3\%$ — a clinically significant improvement.

16. Mathematical Explanation of Class Weights

Class weights are calculated using the following formula to ensure that the training loss reflects the inverse frequency of each class:

$$W_c = N / (C * N_c)$$

Where N is the total number of samples, C is the number of classes, and N_c is the number of samples in class c . For the HAM10000 dataset ($N = 10,015$, $C = 2$ classes):

- Benign Weight (W_0): $W_0 = 10,015 / (2 * 8,061) \approx 0.62$
- Malignant Weight (W_1): $W_1 = 10,015 / (2 * 1,954) \approx 2.56$

The ratio of weights is $2.56 / 0.62 \approx 4.12$. Misclassifying a malignant lesion contributes 4 times more to the training loss gradient than misclassifying a benign lesion, forcing the model to prioritize detection of malignant cases during optimization.

References

1. [Hussein, A. A., & Abdulazeez, A. M. \(2024\). A Review on Utilizing Machine Learning Classification Algorithms for Skin Cancer. Journal of Applied Science and Technology Trends, 5\(2\), 60–71.](#)
2. [Adamu, S. \(2024\). The future of skin cancer diagnosis: a comprehensive systematic literature review of machine learning and deep learning models. Cogent Engineering.](#)
3. [Hermosilla, P., Soto, R., Vega, E., Suazo, C., & Ponce, J. \(2024\). Skin Cancer Detection and Classification Using Neural Network Algorithms: A Systematic Review. Diagnostics, 14\(4\), 454.](#)
4. [Baladhandapani, A., et al. \(2026\). Developing an advanced deep learning framework for skin cancer... Biomedical Signal Processing and Control.](#)
5. [International Agency for Research on Cancer \(IARC\). \(2025\). New study on the global burden of cutaneous melanoma incidence attributable to ultraviolet radiation.](#)
6. [StatPearls. \(2025\). Skin Cancer. NCBI Bookshelf.](#)
7. [World Health Organization. \(2026\). Cancer fact sheet.](#)
8. [Performance evaluation of E-VGG19 model: Enhancing real-time skin cancer detection and classification ☆ Irfan Ali Kandhro Urooj Malik a a , Selvakumar Manickam , Anum Naz a b , Kanwal Fatima , Abdulhalim Dandoush a c , *](#)